INFORME TÉCNICO · 5 NOV 2026

Onoto y Sazón: auditoría del sistema

Foco principal: ¿es correcto el enfoque del control de stock? Repo `webpack-onotoYSazon` · 5 análisis en paralelo (diseño, backend, frontend, navegador, stock) + contraste con referencias en línea.

Veredicto corto: la **idea base es correcta** (stock = libro de movimientos con motivo obligatorio, no un número que se sobrescribe). Pero la implementación **deja fuera del libro a las ventas**, las **reservas no caducan** y **no todas las rutas que consumen stock lo verifican**. Eso abre la puerta a bloquear el inventario, sobrevender y reescribir el historial.

Es un problema de **completar** el modelo, no de reemplazarlo.

1	3	5
CRÍTICO	ALTOS STOCK	MEDIOS STOCK

Todas las pruebas se hicieron sobre copias en un entorno aislado; el repositorio no se modificó.

1. Cómo funciona el stock hoy

La unidad es la **hallaca**. No hay columna de saldo: el stock se calcula siempre al vuelo.

Fórmula actual

`stock = Σ stock_movements.delta − Σ orders.quantity_hallacas`
(pedidos con `status ≠ 'cancelado'` y `stock_exempt = 0`)

backend/controllers/packs.controller.js:23-32

```
ENTRADAS (libro stock_movements)
 ├ Tanda nueva ─tx→ +qty  (kind=tanda)
 └ Ajuste admin ──→ ±qty  (kind=ajuste, motivo)
       │  ⚠ DELETE físico permitido
       ▼
     Σ delta
       −
 Σ pedidos "vivos" (estado ACTUAL de orders)
 ├ Web: tx{ stock ≥ qty ? INSERT : 409 } ✔
 │   pendiente_pago reserva… para siempre ⚠
 ├ Venta manual admin: sin chequeo ⚠
 ├ PATCH estado/cantidad: sin chequeo ⚠
 ├ confirmStock (encargo→stock): tx ✔
 └ Cancelar/borrar: libera (aunque
   ya estuviera entregado) ⚠
       ▼
 GET /packs → available, stock_available
 Kardex admin = movimientos + ventas
   recalculadas desde orders ⚠
```

Lo que está bien

- El stock es **un dato propio con motivo obligatorio** (ADR-3), separado de los costos de las tandas (invariante I-1).
- El pedido web **valida e inserta en la misma transacción**; precio y hallacas salen de la BD, no del cliente.
- Tanda + movimiento de ingreso se crean de forma atómica; `confirmStock` revalida dentro de su

transacción.

- Con un solo proceso Node (better-sqlite3 es síncrono) no hay doble venta en el checkout web.

2. ¿Es correcto el enfoque? Contraste con la práctica recomendada

Se compararon 5 principios que repiten las referencias de diseño de inventarios con lo que hace el código.

A MEDIAS **P1 · El libro de movimientos es la fuente de verdad**

Las guías recomiendan registrar **cada** entrada y salida como fila con signo, tipo y documento origen, y derivar el saldo de ahí.

Aquí: sí para producción y ajustes; **no para las ventas**, que se derivan del estado mutable de `orders`. Resultado: el kardex de días pasados cambia al editar o cancelar un pedido.

Ref: Cleverence (inventory DB), DEV/Pytagotech, Templacity.

NO **P2 · Los asientos son inmutables; se corrige con contra-asiento**

Fowler (*Patterns for Accounting*): el saldo es la suma de entradas inmutables y un error se corrige con una entrada opuesta (*Reversal Adjustment*), nunca editando ni borrando.

Aquí: hay `DELETE` físico de movimientos, pedidos y tandas, y edición directa de `quantity_hallacas`, sin registro de quién ni qué valor tenía antes.

Ref: martinfowler.com/eaaDev/AccountingNarrative.html · AccountingTransaction.html

A MEDIAS **P3 · Distinguir físico, comprometido y disponible**

ERP y Shopify separan **on-hand** (lo que existe), **committed** (reservado a pedidos) y **available/ATP** (lo que se puede prometer).

Aquí: todo se colapsa en un único número. Un `pendiente_pago` anónimo pesa igual que uno pagado, y un pedido **entregado** que se cancela "devuelve" hallacas que ya no existen físicamente.

Ref: Oracle JD Edwards «Types of Quantities», Shopify «Available to promise».

NO **P4 · Las reservas tienen caducidad**

El patrón estándar en e-commerce es reservar al hacer checkout y **liberar automáticamente** si no se paga en X minutos/horas (TTL). Shopify guarda reservas y ledger en la misma BD para que reservar y confirmar sean atómicos.

Aquí: el pedido web reserva sin login y **no caduca nunca**. Esto es el hallazgo crítico C1.

Ref: Shopify Engineering «Scaling inventory reservations», OneUptime «Inventory reservation».

A MEDIAS **P5 · Chequeo + escritura atómicos en TODA ruta que consume**

Dos ventas simultáneas de la última unidad no pueden ambas tener éxito: el chequeo y la escritura deben ser una sola operación. En SQLite se recomienda `BEGIN IMMEDIATE` para tomar el lock de escritura antes de leer.

Aquí: bien en checkout web y `confirmStock`; **falta** en reactivar/editar pedidos y en ventas manuales. Las transacciones usan `BEGIN` diferido (con 2 procesos da 500 en vez de 409).

Ref: Coding Shuttle (LLD inventario), Simon Willison «SQLite busy», tenthousandmeters.com.

Conclusión: el enfoque **ledger + saldo derivado** es exactamente lo que recomiendan Fowler y las guías de inventario; es la decisión correcta para un negocio que necesita saber «por qué» cambió el stock. Lo que falla es que el ledger está **incompleto** (ventas fuera), es **mutable** (deletes/ediciones) y no modela **reservas con vencimiento**.

3. Hallazgos del control de stock

Reproducidos con scripts contra los controladores reales sobre BDs temporales.

CRÍTICO **C1 · Un anónimo puede bloquear todo el inventario**

El checkout no exige login; un pedido `pendiente_pago` reserva en el acto, sin caducidad, hasta 10.000 hallacas por request. Probado: con stock 300, un request `{p10 × 30}` → 201, stock 0 y todos los packs «Agotado».

orders.controller.js:87, 95-137 · orders.routes.js:33

ALTO **A1 · Reactivar o subir cantidad no revisa stock → sobreventa**

Stock 10 → pedido de 8 → cancelado → otro cliente compra 8 → el admin restaura el primero a «pagado» desde el select: **stock −6** sin aviso. Editar la cantidad a 500 deja −498.

admin.controller.js:186-192, 219-226

ALTO **A2 · Sin auditoría; el kardex reescribe la historia**

Las ventas se recalculan desde el estado actual: editar una venta cambió el saldo del 02-nov de 0 a 290; cancelarla hizo desaparecer la línea. Borrados físicos de movimientos, pedidos y tandas sin rastro.

stock.controller.js:51-67, 163 · admin.controller.js:309 · batches.controller.js:183

ALTO **A3 · Cancelar un pedido entregado infla el stock**

Pedido de 3 entregado → cancelado → stock 992 → 995. No existe el concepto de devolución ni merma; la web vuelve a vender hallacas que no existen.

packs.controller.js:26-29

MEDIO **M1–M5**

- **Stock negativo sin confirmación** en egresos y ventas manuales (probado −9905 desde el panel).
- **Fechas futuras aceptadas** (hasta 9999-12-31): una tanda futura se vuelve la «vigente» para todas las ventas.
- **Encargo confirmado** aparece en el kardex con su fecha original → saldos históricos negativos.
- **Zonas horarias mezcladas**: UTC en `orders`, local en movimientos; sin `TZ=America/Santiago` en Render una venta de las 22:30 cae al día siguiente.
- **BEGIN diferido**: con 2 procesos, `SQLITE_BUSY_SNAPSHOT` → 500 en vez de 409.



Panel de Producción a 375 px: un único número de stock, sin separar físico / comprometido / disponible.

4. Modelo de stock propuesto

Mantener ADR-3, pero **completar el libro** y añadir **reservas con vencimiento**. Proporcional a un negocio estacional pequeño: sin Redis ni microservicios, todo en SQLite.

```
stock_movements (inmutable, sin DELETE)
  kind: tanda | ajuste | venta | anulacion
        | devolucion | merma
  order_id, created_by, reason
       ▼
FÍSICO      = Σ delta
COMPROMETIDO = Σ reservas vivas
   (pendiente_pago web con expires_at > now)
DISPONIBLE  = FÍSICO − COMPROMETIDO
       ▼
pagar/entregar → reserva se convierte
                 en movimiento 'venta'
vence TTL     → reserva se libera sola
cancelar      → movimiento 'anulacion'
cancelar entregado → elegir:
                 'devolucion' o 'merma'
```

Reglas

- **Toda** operación que aumente el consumo (crear, reactivar, subir cantidad, venta manual) pasa por una única función `reservar(qty)` dentro de `db.transaction(...).immediate()`.
- Corregir = registrar contra-asiento. Los «borrados» pasan a `deleted_at` + movimiento inverso.
- El admin puede forzar stock negativo solo con `force:true` y confirmación explícita en la UI («registrar la realidad»).
- Fechas: rechazar futuras (+1 día de margen), fijar `process.env.TZ='America/Santiago'` al arrancar.

El kardex pasa a ser una simple lectura ordenada del libro: deja de recalcularse y nunca cambia hacia atrás.

5. Plan de acción (stock)

Fase 1 · urgente, cambios pequeños

1. **TTL a la reserva web**: `reserved_until` en pedidos web `pendiente_pago` (p. ej. 24–48 h, configurable), excluir vencidos del cálculo. Bajar el tope por pedido (≈200 hallacas) y rate-limit propio en `POST /orders` por IP + teléfono.
2. **Chequeo de stock en `updateOrder`** cuando el delta de reserva sea positivo; 409 salvo `force:true`.
3. **Cancelar entregado** obliga a elegir devolución o merma; bloquear `delivered` en cancelados.
4. `.immediate()` en transacciones de escritura, `busy_timeout`, y mapear `SQLITE_BUSY*` a 409/503.

Fase 2 · completar el ledger

5. Ventas, anulaciones y confirmaciones escriben en `stock_movements` con `order_id`. Tabla `order_events` (antes/después, usuario).
6. Eliminar `DELETE` físicos → soft delete + contra-asiento.
7. Validar fechas (sin futuras), TZ fija, encargos fechados al confirmarse.

Fase 3 · red de seguridad

8. Tests con `node:test` + BD `:memory:` para cada escenario de este informe (hoy no hay ningún test ni CI) y **backup diario** del archivo SQLite.

6. Resto del sistema (resumen)

Arquitectura y deploy

- C Si falta `API_URL` (con sufijo `/api`) en Vercel, el rewrite devuelve HTML a cada llamada: catálogo falso, sin pedidos ni login. Fallar el build si falta.
- A Una sola BD SQLite en disco de Render **sin backups**; obliga a una única instancia.
- A Sin tests ni CI; los invariantes solo están protegidos por comentarios.
- M Esquema duplicado (`schema.sql` vs `migrate.js`, sin `stock_exempt`) y migraciones detectadas por texto: usar `PRAGMA user_version`.
- M ADR-3, plan 09 y auditoría 10 citados en el código pero excluidos del repo por `.gitignore`.

Backend

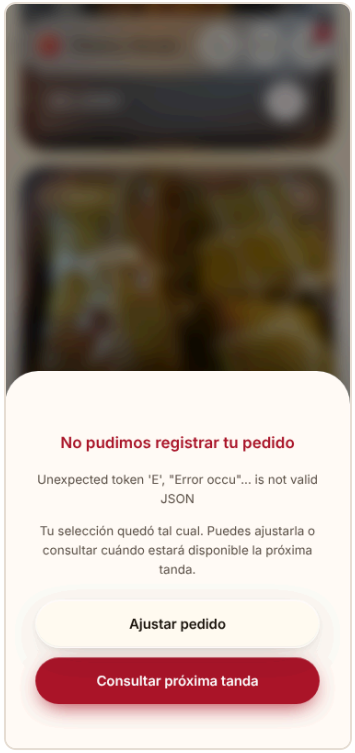
- A Sin `trust proxy`: en Render todos comparten IP → 100 req/15 min para todo el sitio y 5 fallos de login de cualquiera bloquean al admin (verificado).
- A El limitador de auth cuenta también errores de validación (400/409).
- M JSON malformado, body grande y CORS → 500 en vez de 400/413/403.
- M Números sin tope: `totalClp: 1e300` rompe todos los reportes. Tipos inesperados → 500.
- M `.escape()` guarda entidades HTML (`O'Higgins`). Faltan índices (`order_items.order_id`…).
- B Sin validar `JWT_SECRET`/`ADMIN_PASSWORD` al arrancar; rol leído del JWT y no de la BD.

Frontend

- A Token vencido no cierra sesión (no se maneja 401).
- A El modo oscuro guardado se pisa en cada carga (`App.jsx:82-86`).
- A Modales sin gestión de foco (accesibilidad de teclado).
- M Panel admin (1.170 líneas) dentro del bundle público: cargar con `React.lazy`.
- M Imágenes: `tradition-hands.jpeg` 435 KB, 4,7 MB de PNG sin uso, packs en `i.ibb.co`, `favicon.ico` vacío.
- B Build OK: ~85 KB JS gzip, 0 vulnerabilidades en dependencias de producción.

7. Pruebas en navegador

Back + front levantados localmente y recorridos con Playwright a 375 px y 1280 px.



Backend caído (o 502 en arranque en frío): el cliente ve el error técnico crudo.

Hallazgos

- A Notas y dirección se muestran escapadas en el admin (`<b>`).
- A Backend caído/502 → «Unexpected token … is not valid JSON» y catálogo de respaldo con disponibilidad falsa.
- A Rate limit agotado en una sola sesión de pruebas.

- M Stock −9905 tras un egreso sin confirmación; leyenda engañosa («registraste más ventas…»).
- M Registro con clave débil solo dice «Datos inválidos»; selects de Ventas truncados; tablas admin a 375 px con scroll oculto; un error de validación borra la tabla de Packs.
- B Moneda negativa «$-137.000», objetivos táctiles < 44 px, botón de WhatsApp tapa contenido.

Funciona bien

- Sin errores de consola ni overflow horizontal en la web pública.
- Pedido web: 201, descuenta stock (94→90), aparece en admin y abre WhatsApp.
- Las 6 pestañas del admin operan; endpoints protegidos devuelven 401 sin token.

8. Prioridades globales

1. **Stock Fase 1** (TTL de reservas, chequeo en edición, devoluciones/merma, `.immediate()`).
2. **Deploy seguro**: `API_URL` obligatoria, `trust proxy`, `TZ`, validación de variables, cambiar la clave admin de ejemplo.
3. **Backup diario** de SQLite.
4. **Cliente HTTP robusto**: parseo seguro, timeout, 401 → cerrar sesión.
5. Quitar `.escape()`, validar números/fechas/tipos, handler de errores 4xx.
6. **Stock Fase 2**: ventas al ledger, sin deletes físicos.
7. Tests + CI, índices, `user_version`.
8. Frontend: lazy admin, foco en modales, imágenes, tema oscuro.

Referencias consultadas

1. Fowler — Patterns for Accounting: martinfowler.com/eaaDev/AccountingNarrative.html
2. Fowler — Accounting Transaction: martinfowler.com/eaaDev/AccountingTransaction.html
3. Shopify Engineering — Scaling inventory reservations: shopify.engineering/scaling-inventory-reservations
4. Shopify — Available to promise: shopify.com/blog/available-to-promise
5. Oracle JD Edwards — Types of Quantities: docs.oracle.com/en/applications/jd-edwards/supply-chain-manufacturing/9.2/eoaim/types-of-quantities.html
6. Cleverence — Database for inventory: cleverence.com/articles/for-business/database-for-inventory-4827/
7. Cleverence — Inventory database in SQL: cleverence.com/articles/for-business/how-to-create-an-inventory-database-in-sql-4732/
8. DEV — Inventory software starts with movement history: dev.to/pytagotech/inventory-software-starts-with-movement-history-onf
9. Coding Shuttle — Design an inventory management system: codingshuttle.com/low-level-system-design-lld-handbook/design-an-i

nventory-management-system

10. OneUptime — Inventory reservation with Redis: oneuptime.com/blog/post/2026-03-31-redis-inventory-reservation/view
11. Simon Willison — SQLite busy: simonwillison.net/2025/Feb/17/sqlite-busy
12. SQLite concurrent writes: tenthousandmeters.com/blog/sqlite-concurrent-writes-and-database-is-locked-errors/